



SPOŁECZNA AKADEMIA NAUK

www.san.edu.pl

Kierunek studiów: Informatyka

Nazwisko i Imię: Roman Piotr

Numer albumu: 122217

Grupa: IV

nr grupy: 4

Studia Inż. Zaoczne

Bazy danych i aplikacje – inż. sem. 6

Sprawozdanie

z projektu

**Wirtualny Dziekanat – aplikacja webowa ASP.NET
Core MVC z bazą danych i wzorcem CQRS**

Prowadzący zajęcia: **mgr inż. Andrzej Herman**

Łódź 2025/2026

Spis treści

1	Wstęp	2
2	Cel projektu	3
3	Przegląd istniejących rozwiązań	4
3.1	Rozwiązanie A – USOS	4
3.2	Rozwiązanie B – komercyjne systemy dziekanatowe	4
3.3	Rozwiązanie C – arkusz kalkulacyjny	4
3.4	Porównanie	4
4	Opis architektury	6
4.1	Wykorzystane technologie	6
4.2	Warstwy aplikacji	6
4.3	Wzorzec CQRS	7
4.4	Struktura projektu	8
5	Opis implementacji	9
5.1	Baza danych	9
5.2	Backend	11
5.2.1	Interfejsy CQRS	11
5.2.2	Przykładowa komenda	11
5.2.3	Przykładowe zapytanie	12
5.2.4	Kontrolery i wstrzykiwanie zależności	12
5.3	Frontend	14
5.4	Testy	18
6	Uruchomienie	20
7	Podsumowanie i wnioski	21
	Bibliografia	24

1 Wstęp

Obsługa toku studiów – ewidencja studentów, lista prowadzonych przedmiotów i wystawianie ocen – to jeden z najbardziej typowych przykładów aplikacji bazodanowej. Dane mają wyraźną strukturę relacyjną (student, przedmiot i łącząca je ocena), a użytkownik wykonuje na nich zarówno operacje odczytu (przeglądanie list, średnia ocen), jak i zapisu (dodawanie, edycja, usuwanie).

Przedmiotem sprawozdania jest aplikacja **Wirtualny Dziekanat**, zrealizowana w technologii ASP.NET Core MVC z bazą danych SQLite obsługiwaną przez Entity Framework Core [1, 2]. Logika aplikacji została podzielona zgodnie ze wzorcem **CQRS** (*Command Query Responsibility Segregation*), który oddziela operacje odczytu danych od operacji ich modyfikacji [3].

W kolejnych rozdziałach przedstawiono cel projektu, krótki przegląd istniejących rozwiązań, architekturę aplikacji, szczegóły implementacji (baza danych, warstwa logiki, interfejs użytkownika), sposób testowania i uruchomienia oraz wnioski.

2 Cel projektu

Celem projektu było stworzenie prostej aplikacji webowej typu *wirtualny dziekanat*, która pozwala pracownikowi dziekanatu zarządzać studentami, przedmiotami i ocenami, przechowując dane w relacyjnej bazie danych. Dodatkowym celem było praktyczne zastosowanie wzorca CQRS bez korzystania z gotowych bibliotek.

Wymagania funkcjonalne zestawiono w tabeli 1.

Tabela 1: Wymagania funkcjonalne aplikacji

Nr	Wymaganie
F1	Wyświetlanie listy studentów, dodawanie, edycja i usuwanie studenta.
F2	Wyszukiwanie studentów po nazwisku lub numerze albumu.
F3	Wyświetlanie listy przedmiotów, dodawanie, edycja i usuwanie przedmiotu.
F4	Wystawianie oceny (student, przedmiot, ocena ze skali 2.0–5.0, data) i usuwanie oceny.
F5	Podgląd ocen wybranego studenta wraz z wyliczoną średnią.
F6	Strona główna ze statystykami: liczba studentów, przedmiotów i ocen.
F7	Usunięcie studenta lub przedmiotu usuwa również powiązane z nim oceny.
F8	Walidacja danych w formularzach z komunikatami w języku polskim.

Wymagania нефункциональные:

- aplikacja działa w przeglądarce, a interfejs użytkownika jest w języku polskim,
- kod źródłowy (nazwy klas, metod, właściwości i komentarze) jest napisany w języku angielskim,
- baza danych nie wymaga instalowania serwera – jest to jeden plik tworzony automatycznie przy pierwszym uruchomieniu,
- kontrolery nie korzystają bezpośrednio z bazy danych, tylko z klas komend i zapytań (CQRS),
- projekt można otworzyć i uruchomić w Visual Studio 2026 bez dodatkowej konfiguracji.

3 Przegląd istniejących rozwiązań

Funkcje wirtualnego dziekanatu są na uczelniach realizowane przez rozbudowane systemy informatyczne, ale w małej skali (np. pojedynczy prowadzący) często wystarcza arkusz kalkulacyjny. Poniżej krótko omówiono trzy typowe podejścia.

3.1 Rozwiązanie A – USOS

USOS (Uniwersytecki System Obsługi Studiów) jest rozwijany przez Międzyuniwersyteckie Centrum Informatyzacji i używany przez wiele polskich uczelni [4]. Obejmuje pełen tok studiów: rekrutację, programy studiów, rejestracje na zajęcia, protokoły ocen, stypendia i dyplomy. Studenci i pracownicy korzystają z niego przez przeglądarkę (moduł USOSweb). Zaletą jest kompletność i zgodność z przepisami, wadą – duża złożoność wdrożenia i utrzymania, zupełnie nieadekwatna do prostych zastosowań.

3.2 Rozwiązanie B – komercyjne systemy dziekanatowe

Część uczelni, zwłaszcza niepublicznych, korzysta z komercyjnych systemów obsługi dziekanatu, zwykle z modułem *wirtualnego dziekanatu* dla studentów (podgląd ocen, płatności, ogłoszeń). Są to rozwiązania gotowe do użycia, z pomocą techniczną producenta, ale płatne i słabo modyfikowalne przez samą uczelnię.

3.3 Rozwiązanie C – arkusz kalkulacyjny

Najprostszym sposobem prowadzenia ewidencji ocen jest arkusz kalkulacyjny (np. Excel). Nie wymaga on programowania, ale nie zapewnia integralności danych: ta sama osoba może być wpisana dwa razy w różny sposób, usunięcie studenta nie usuwa jego ocen, a nad poprawnością wartości (np. oceny spoza skali) trzeba panować ręcznie.

3.4 Porównanie

Zestawienie cech rozwiązań zawiera tabela 2. Projektowana aplikacja plasuje się pomiędzy arkuszem a pełnym systemem: jest prosta, ale korzysta z relacyjnej bazy danych, więc zapewnia spójność danych i walidację.

Tabela 2: Porównanie istniejących rozwiązań z projektowaną aplikacją

Cecha	USOS	Komercyjne	Arkusz	Projekt
Relacyjna baza danych	tak	tak	nie	tak
Walidacja danych	tak	tak	ręczna	tak
Prostota wdrożenia	nie	częściowo	tak	tak
Brak kosztów licencji	nie ¹	nie	zależy	tak
Pełny tok studiów	tak	tak	nie	nie

¹Uczelnia ponosi koszty członkostwa w konsorcjum i utrzymania systemu.

4 Opis architektury

4.1 Wykorzystane technologie

Technologie użyte w projekcie zestawiono w tabeli 3.

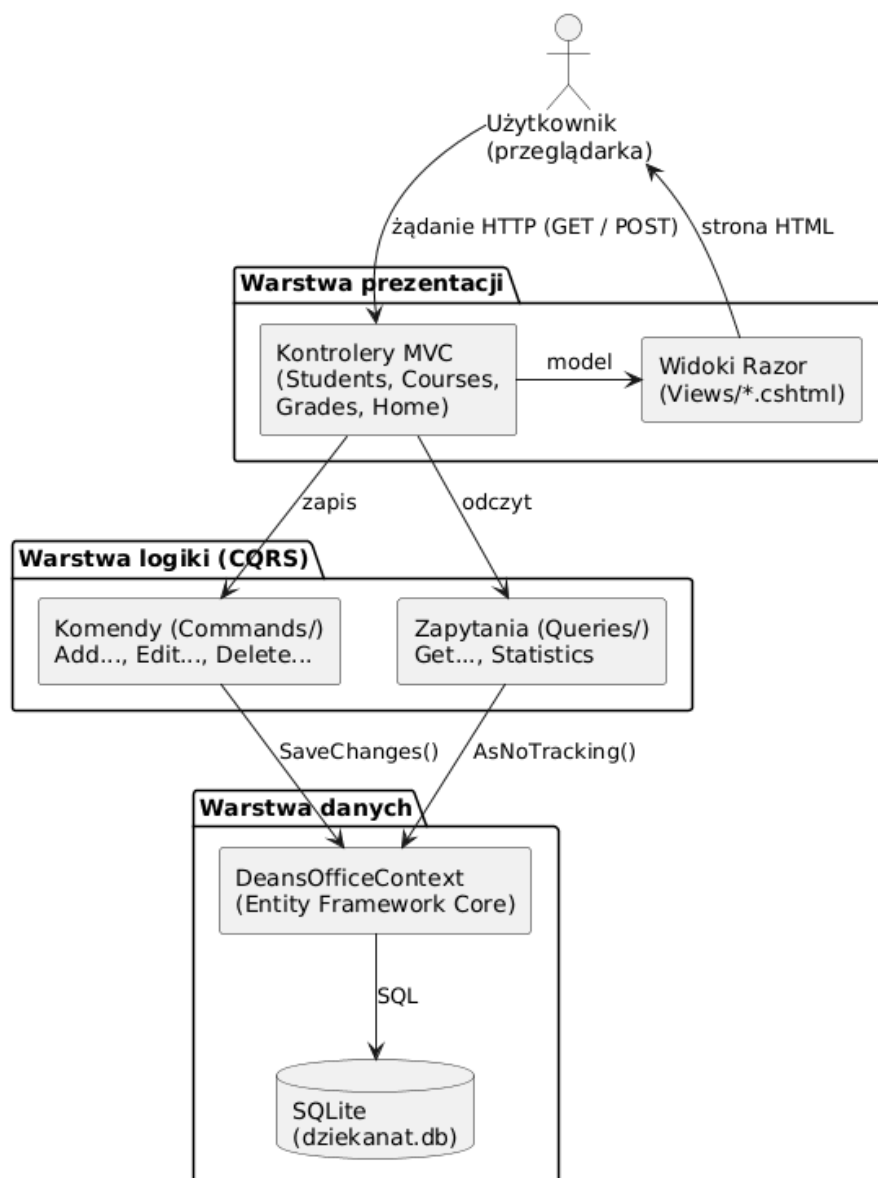
Tabela 3: Technologie wykorzystane w projekcie

Technologia	Zastosowanie
C# / .NET 8	język programowania i platforma uruchomieniowa
ASP.NET Core MVC	framework aplikacji webowej (wzorzec Model–View–Controller) [1]
Entity Framework Core 8	ORM – mapowanie klas C# na tabele bazy danych [2]
SQLite	relacyjna baza danych zapisywana w jednym pliku, bez serwera
Razor	silnik widoków (pliki .cshtml)
Bootstrap 5.1	style CSS interfejsu użytkownika
Visual Studio 2026	środowisko programistyczne

4.2 Warstwy aplikacji

Aplikacja składa się z trzech warstw (rys. 1):

- **warstwa prezentacji** – kontrolery MVC przyjmują żądania HTTP i przekazują model do widoków Razor, które generują stronę HTML,
- **warstwa logiki** – klasy komend (Commands/) i zapytań (Queries/) wraz z ich handlerami, zgodnie ze wzorcem CQRS,
- **warstwa danych** – kontekst bazy DeansOfficeContext (Entity Framework Core) i plik bazy SQLite dziekanat.db.



Rysunek 1: Architektura aplikacji Wirtualny Dziekanat

4.3 Wzorzec CQRS

CQRS polega na rozdzieleniu modelu na dwie części: **komendy** (*commands*), które zmieniają stan systemu i niczego nie zwracają, oraz **zapytania** (*queries*), które zwracają dane, ale niczego nie zmieniają [3]. W dużych systemach obie strony mogą mieć nawet osobne bazy danych; w tym projekcie zastosowano wersję podstawową – wspólna baza, ale osobne klasy dla każdej operacji odczytu i zapisu.

Każda operacja składa się z dwóch klas: klasy z danymi (np. `AddStudentCommand`) oraz handlera, który ją wykonuje (np. `AddStudentHandler`). Wszystkie operacje w projekcie przedstawia tabela 4.

Tabela 4: Zapytania i komendy w projekcie

Klasa	Działanie
Zapytania (odczyt)	
GetStudentsQuery	lista studentów, opcjonalnie filtrowana (Search)
GetStudentQuery	jeden student wraz z ocenami
GetCoursesQuery	lista przedmiotów
GetCourseQuery	jeden przedmiot
GetGradesQuery	lista ocen ze studentem i przedmiotem
StatisticsQuery	liczba studentów, przedmiotów i ocen
Komendy (zapis)	
AddStudentCommand	dodanie studenta
EditStudentCommand	edycja danych studenta
DeleteStudentCommand	usunięcie studenta razem z jego ocenami
AddCourseCommand	dodanie przedmiotu
EditCourseCommand	edycja przedmiotu
DeleteCourseCommand	usunięcie przedmiotu razem z ocenami
AddGradeCommand	wystawienie oceny
DeleteGradeCommand	usunięcie oceny

4.4 Struktura projektu

Pliki projektu są pogrupowane w katalogi odpowiadające warstwom (listing 1).

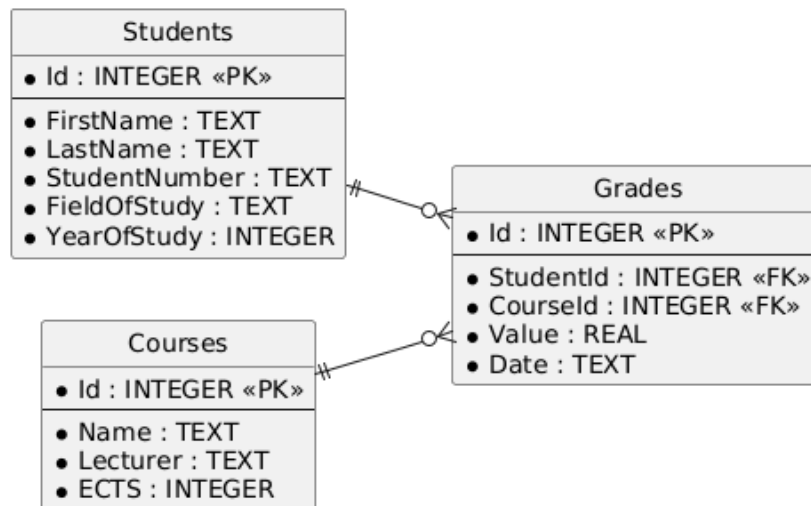
Listing 1: Struktura katalogów projektu

WirtualnyDziekanat/	
Commands/	komendy - zapis danych (Add..., Edit..., Delete...)
Queries/	zapytania - odczyt danych (Get..., Statistics)
CQRS/	interfejsy ICommandHandler i IQueryHandler
Controllers/	kontrolery MVC
Data/	DeansOfficeContext (Entity Framework Core)
Models/	klasy encji: Student, Course, Grade
Views/	widoki Razor (interfejs po polsku)
wwwroot/	pliki statyczne (Bootstrap, CSS)
Program.cs	konfiguracja aplikacji i dane startowe
appsettings.json	connection string do bazy

5 Opis implementacji

5.1 Baza danych

Baza danych składa się z trzech tabel: Students, Courses i Grades (rys. 2). Tabela Grades łączy studentów z przedmiotami – jeden student może mieć wiele ocen i z jednego przedmiotu może być wystawionych wiele ocen (dwie relacje 1:N).



Rysunek 2: Diagram związków encji bazy danych

Szczegółowy opis kolumn zawierają tabele 5–7. Typy podano tak, jak zapisuje je SQLite; Entity Framework mapuje string na TEXT, int na INTEGER, double na REAL, a datę (DateTime) zapisuje jako tekst.

Tabela 5: Tabela Students

Kolumna	Typ	Opis
Id	INTEGER	klucz główny (autoinkrementacja)
FirstName	TEXT	imię, wymagane
LastName	TEXT	nazwisko, wymagane
StudentNumber	TEXT	numer albumu, wymagany
FieldOfStudy	TEXT	kierunek studiów, wymagany
YearOfStudy	INTEGER	rok studiów, zakres 1–5

Tabela 6: Tabela Courses

Kolumna	Typ	Opis
Id	INTEGER	klucz główny (autoinkrementacja)
Name	TEXT	nazwa przedmiotu, wymagana
Lecturer	TEXT	prowadzący, wymagany
ECTS	INTEGER	liczba punktów ECTS, zakres 1–30

Tabela 7: Tabela Grades

Kolumna	Typ	Opis
Id	INTEGER	klucz główny (autoinkrementacja)
StudentId	INTEGER	klucz obcy do Students
CourseId	INTEGER	klucz obcy do Courses
Value	REAL	ocena, zakres 2.0–5.0
Date	TEXT	data wystawienia oceny

Połączenie z bazą realizuje klasa `DeansOfficeContext` dziedzicząca po `DbContext`. Każda właściwość typu `DbSet` odpowiada jednej tabeli (listing 2).

Listing 2: Kontekst bazy danych

```

1 public class DeansOfficeContext : DbContext
2 {
3     public DeansOfficeContext(DbContextOptions<DeansOfficeContext>
4         options) : base(options)
5     {
6     }
7     public DbSet<Student> Students { get; set; }
8     public DbSet<Course> Courses { get; set; }
9     public DbSet<Grade> Grades { get; set; }
10 }
```

Ścieżka do pliku bazy znajduje się w `appsettings.json` ("DeansOffice": "Data Source=dziekanat.db"). SQLite interpretuje ścieżkę względną względem bieżącego katalogu procesu, dlatego w `Program.cs` jest ona zamieniana na ścieżkę w katalogu projektu (`ContentRootPath`) – dzięki temu aplikacja zawsze korzysta z tego samego pliku, niezależnie od tego, skąd zostanie uruchomiona. Przy starcie aplikacji wywoływana jest metoda `EnsureCreated()`, która tworzy bazę i tabele, jeśli jeszcze nie istnieją [2]. Jeżeli tabela studentów jest pusta, `Program.cs` dodaje przykładowe dane (trzech studentów, trzy przedmioty i cztery oceny), dzięki czemu aplikacja od razu nadaje się do prezentacji.

Na podstawie właściwości nawigacyjnych (`Grade.Student`, `Grade.Course`) Entity Framework sam tworzy klucze obce z regułą `ON DELETE CASCADE` oraz indeksy na kolumnach `StudentId` i `CourseId`. Mimo to komendy usuwające studenta lub przedmiot jawnie usuwają

najpierw powiązane oceny, żeby zachowanie aplikacji było widoczne w kodzie i nie zależało od ustawień bazy.

5.2 Backend

5.2.1 Interfejsy CQRS

Podstawą implementacji są dwa generyczne interfejsy (listing 3). Handler komendy przyjmuje komendę i nic nie zwraca, handler zapytania przyjmuje zapytanie i zwraca wynik określonego typu.

Listing 3: Interfejsy CQRS (CQRS/Interfaces.cs)

```
1 // Command - changes data in the database, returns nothing
2 public interface ICommandHandler<TCommand>
3 {
4     void Handle(TCommand command);
5 }
6
7 // Query - only reads data, changes nothing
8 public interface IQueryHandler<TQuery, TResult>
9 {
10     TResult Handle(TQuery query);
11 }
```

5.2.2 Przykładowa komenda

Listing 4 przedstawia komendę edycji studenta. Handler pobiera encję metodą Find, zmienia jej pola i zapisuje zmiany. Entity Framework śledzi pobrany obiekt, więc sam generuje odpowiednie polecenie UPDATE.

Listing 4: Komenda edycji studenta (Commands/EditStudent.cs)

```
1 public class EditStudentCommand
2 {
3     public int Id { get; set; }
4     public string FirstName { get; set; }
5     public string LastName { get; set; }
6     public string StudentNumber { get; set; }
7     public string FieldOfStudy { get; set; }
8     public int YearOfStudy { get; set; }
9 }
10
11 public class EditStudentHandler : ICommandHandler<EditStudentCommand>
12 {
13     private readonly DeansOfficeContext _context;
14
15     public EditStudentHandler(DeansOfficeContext context)
16     {
17         _context = context;
18     }
19 }
```

```

20     public void Handle(EditStudentCommand command)
21     {
22         var student = _context.Students.Find(command.Id);
23         if (student == null)
24         {
25             return;
26         }
27
28         student.FirstName = command.FirstName;
29         student.LastName = command.LastName;
30         student.StudentNumber = command.StudentNumber;
31         student.FieldOfStudy = command.FieldOfStudy;
32         student.YearOfStudy = command.YearOfStudy;
33         _context.SaveChanges();
34     }
35 }

```

5.2.3 Przykładowe zapytanie

Zapytania używają metody `AsNoTracking()` – skoro dane są tylko wyświetlane, Entity Framework nie musi śledzić zmian w pobranych obiektach, co zmniejsza narzut [2]. Listing 5 pokazuje zapytanie zwracające listę ocen razem z danymi studenta i przedmiotu (`Include`), posortowaną od najnowszej.

Listing 5: Zapytanie o listę ocen (`Queries/GetGrades.cs`)

```

1  public class GetGradesHandler : IQueryHandler<GetGradesQuery, List<Grade
    >>
2  {
3      private readonly DeansOfficeContext _context;
4
5      public GetGradesHandler(DeansOfficeContext context)
6      {
7          _context = context;
8      }
9
10     public List<Grade> Handle(GetGradesQuery query)
11     {
12         return _context.Grades.AsNoTracking()
13             .Include(g => g.Student)
14             .Include(g => g.Course)
15             .OrderByDescending(g => g.Date)
16             .ToList();
17     }
18 }

```

5.2.4 Kontrolery i wstrzykiwanie zależności

Kontrolery nie znają kontekstu bazy danych – otrzymują w konstruktorze potrzebne handlersy i tylko je wywołują. Handlersy są rejestrowane w kontenerze wstrzykiwania zależności w `Program.cs` z czasem życia `Scoped`, czyli jedna instancja na żądanie HTTP, tak samo jak

DbContext (listing 6).

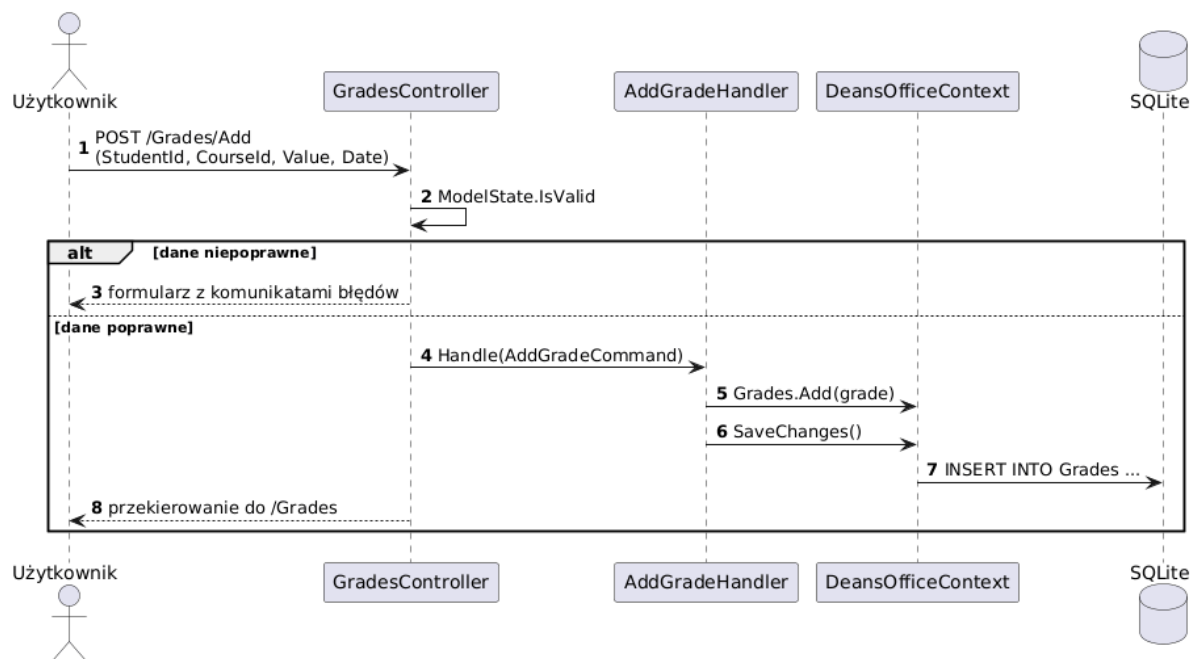
Listing 6: Rejestracja handlerów w Program.cs (fragment)

```
1 // CQRS - queries (read)
2 builder.Services.AddScoped<GetStudentsHandler>();
3 builder.Services.AddScoped<GetStudentHandler>();
4 // ...
5 // CQRS - commands (write)
6 builder.Services.AddScoped<AddStudentHandler>();
7 builder.Services.AddScoped<EditStudentHandler>();
8 // ...
```

Listing 7 przedstawia akcję wystawienia oceny. Jeżeli dane z formularza nie przechodzą walidacji, formularz jest wyświetlany ponownie z komunikatami błędów; w przeciwnym razie kontroler tworzy komendę, przekazuje ją do handlera i przekierowuje użytkownika na listę ocen. Przebieg tej operacji pokazuje diagram sekwencji na rys. 3.

Listing 7: Akcja wystawienia oceny (Controllers/GradesController.cs)

```
1 [HttpPost]
2 public IActionResult Add(Grade grade)
3 {
4     if (!ModelState.IsValid)
5     {
6         FillSelectLists();
7         return View(grade);
8     }
9
10    _addGrade.Handle(new AddGradeCommand
11    {
12        StudentId = grade.StudentId,
13        CourseId = grade.CourseId,
14        Value = grade.Value,
15        Date = grade.Date
16    });
17    return RedirectToAction("Index");
18 }
```



Rysunek 3: Diagram sekwencji – wystawienie oceny

5.3 Frontend

Interfejs użytkownika tworzą widoki Razor (Views/) ze wspólnym szablonem `_Layout.cshtml`, który zawiera pasek nawigacji z odnośnikami do list studentów, przedmiotów i ocen. Wygląd zapewnia biblioteka Bootstrap. Cały tekst widoczny dla użytkownika jest po polsku, mimo że nazwy właściwości w kodzie są angielskie – polskie etykiety pól i komunikaty walidacji zdefiniowano atrybutami w klasach modelu (listing 8) [1].

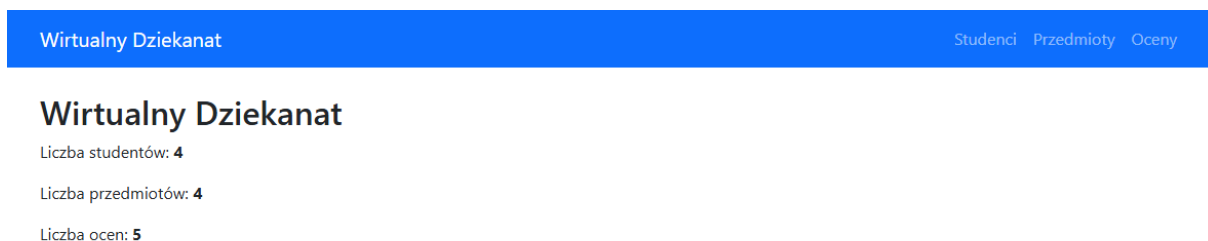
Listing 8: Atrybuty walidacji i etykiety pól (Models/Student.cs, fragment)

```

1 [Required(ErrorMessage = "Pole {0} jest wymagane.")]
2 [Display(Name = "Imię")]
3 public string FirstName { get; set; }
4
5 [Range(1, 5, ErrorMessage = "Pole {0} musi być w zakresie od {1} do {2}.")]
6 [Display(Name = "Rok studiów")]
7 public int YearOfStudy { get; set; } = 1;
  
```

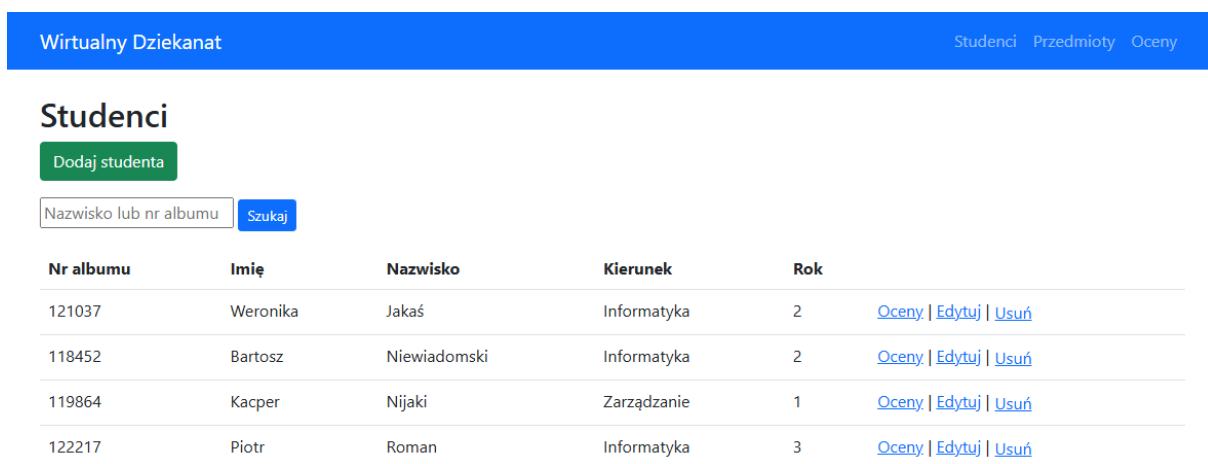
Poniżej przedstawiono kolejne ekrany aplikacji.

Strona główna (rys. 4) wyświetla statystyki pobierane zapytaniem `StatisticsQuery`.

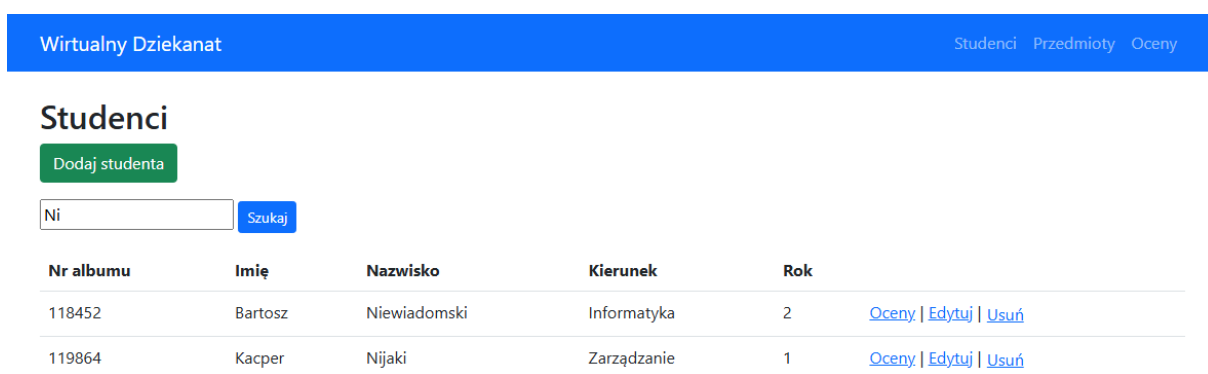


Rysunek 4: Strona główna ze statystykami

Lista studentów (rys. 5) zawiera przycisk dodawania, wyszukiwarkę oraz przy każdym studencie odnośniki do jego ocen, edycji i usunięcia. Usunięcie wymaga potwierdzenia w oknie dialogowym przeglądarki. Wyszukiwarka filtruje studentów po fragmencie nazwiska lub numeru albumu (rys. 6).



Rysunek 5: Lista studentów



Rysunek 6: Wyniki wyszukiwania studentów dla frazy „Ni”

Oceny studenta (rys. 7) – strona szczegółów pokazuje dane studenta, listę jego ocen oraz średnią liczoną w widoku.

Piotr Roman

Nr albumu: 122217, kierunek: Informatyka, rok: 3

Przedmiot	Ocena	Data
Bazy danych i aplikacje	5.0	26.09.2026

Średnia: **5.00**

[Powrót](#)

Rysunek 7: Oceny wybranego studenta ze średnią

Formularze dodawania i edycji studenta (rys. 8) mają te same pola. Po wysłaniu niepoprawnych danych formularz wraca z komunikatami przy polach, których dotyczy błąd (rys. 9).

Edytuj studenta

Imię

Nazwisko

Nr albumu

Kierunek

Rok studiów

[Powrót](#)

Rysunek 8: Formularz edycji studenta

Dodaj studenta

Imię

Pole Imię jest wymagane.

Nazwisko

Pole Nazwisko jest wymagane.

Nr albumu

Pole Nr albumu jest wymagane.

Kierunek

Pole Kierunek jest wymagane.

Rok studiów

Pole Rok studiów musi być w zakresie od 1 do 5.

Zapisz

[Powrót](#)

Rysunek 9: Komunikaty walidacji przy dodawaniu studenta

Lista przedmiotów (rys. 10) pokazuje nazwę, prowadzącego i liczbę punktów ECTS; przedmioty można dodawać, edytować i usuwać.

Przedmioty

Dodaj przedmiot

Nazwa	Prowadzący	ECTS	
Bazy danych	dr Monika Sobczak	5	Edytuj Usuń
Bazy danych i aplikacje	mgr inż. Andrzej Herman	4	Edytuj Usuń
Matematyka dyskretna	prof. Tadeusz Kurek	4	Edytuj Usuń
Programowanie obiektowe	dr Grzegorz Pawlak	6	Edytuj Usuń

Rysunek 10: Lista przedmiotów

Lista ocen (rys. 11) zawiera wszystkie wystawione oceny, od najnowszej. Ocenę wystawia się w formularzu (rys. 12), w którym student i przedmiot wybierane są z list rozwijanych, ocena z listy dozwolonych wartości (2.0, 3.0, 3.5, 4.0, 4.5, 5.0), a data domyślnie ustawiona jest na dzień bieżący.

Oceny

Wystaw ocenę

Student	Przedmiot	Ocena	Data	
Bartosz Niewiadomski	Programowanie obiektowe	4.5	26.09.2026	Usuń
Bartosz Niewiadomski	Bazy danych	4.0	26.09.2026	Usuń
Weronika Jakaś	Programowanie obiektowe	5.0	26.09.2026	Usuń
Kacper Nijaki	Matematyka dyskretna	3.0	26.09.2026	Usuń
Piotr Roman	Bazy danych i aplikacje	5.0	26.09.2026	Usuń

Rysunek 11: Lista wystawionych ocen

Wystaw ocenę

Student

Jakaś Weronika (121037)

Przedmiot

Bazy danych

Ocena

2.0

Data

26.09.2026



Zapisz

[Powrót](#)

Rysunek 12: Formularz wystawiania oceny

5.4 Testy

Aplikację przetestowano ręcznie oraz poleceniem curl, sprawdzając kody odpowiedzi HTTP i treść stron. Scenariusze modyfikujące dane (T6–T8) wykonano na kopii bazy, aby nie zmieniać danych prezentowanych na zrzutach ekranu. Wykonane scenariusze i ich wyniki zestawiono w tabeli 8.

Tabela 8: Scenariusze testowe

Nr	Scenariusz i oczekiwany wynik	Wynik
T1	Wszystkie strony (listy, szczegóły, formularze dodawania i edycji) zwracają kod 200.	OK
T2	Strona główna pokazuje liczby zgodne z zawartością bazy.	OK
T3	Wyszukanie frazy „Ni” zwraca tylko studentów, których nazwisko lub numer albumu ją zawiera.	OK
T4	Wysłanie pustego formularza studenta z rokiem studiów 7 – formularz wraca z polskimi komunikatami przy każdym polu.	OK
T5	Formularz wystawiania oceny ma domyślnie ustawioną bieżącą datę, a lista studentów pokazuje nazwisko, imię i nr albumu.	OK
T6	Wystawienie oceny – nowa ocena pojawia się na liście ocen i na stronie studenta, średnia jest przeliczona.	OK
T7	Usunięcie studenta usuwa także jego oceny (liczba ocen na stronie głównej maleje).	OK
T8	Edycja przedmiotu – nowa nazwa jest widoczna na liście przedmiotów i na liście ocen.	OK

6 Uruchomienie

Wymagania: Visual Studio 2026 z obciążeniem *ASP.NET and web development* oraz zestawem .NET 8 SDK (lub nowszym).

1. Otworzyć plik `WirtualnyDziekanat.csproj` w Visual Studio 2026.
2. Visual Studio samo pobierze z repozytorium NuGet wymagany pakiet Entity Framework Core dla SQLite (`Microsoft.EntityFrameworkCore.Sqlite`).
3. Wybrać profil uruchomieniowy `http` i uruchomić projekt klawiszem F5.
4. Aplikacja otworzy się w przeglądarce pod adresem `http://localhost:5268`.

Alternatywnie projekt można uruchomić z wiersza poleceń, w katalogu projektu: `dotnet run`. Plik bazy `dziekanat.db` zostanie utworzony automatycznie i wypełniony przykładowymi danymi.

7 Podsumowanie i wnioski

W ramach projektu powstała aplikacja webowa realizująca podstawowe funkcje dziekanatu: ewidencję studentów i przedmiotów oraz wystawianie ocen. Wszystkie wymagania funkcjonalne z tabeli 1 zostały spełnione.

Wnioski:

- Entity Framework Core znacznie upraszcza pracę z bazą – schemat tworzony jest z klas modelu, a zapytania pisze się w LINQ, bez ręcznego SQL. Warto jednak kontrolować, co faktycznie trafia do bazy, np. dołączać powiązane dane przez `Include` i wyłączać śledzenie zmian przy odczycie.
- Wzorzec CQRS nawet w podstawowej postaci porządkuje kod: każda operacja jest w osobnej, krótkiej klasie, a kontrolery tylko przekazują dane do handlerów. Kosztem jest większa liczba plików, co przy tak małej aplikacji jest odczuwalne.
- Rozdzielenie języka kodu (angielski) od języka interfejsu (polski) przez atrybuty `Display` i komunikaty walidacji pozwala zachować spójne nazewnictwo w kodzie bez utraty polskiego interfejsu.
- SQLite dobrze sprawdza się w projekcie dydaktycznym – nie wymaga serwera, a cała baza jest jednym plikiem.

Możliwe kierunki rozwoju:

- logowanie i role użytkowników (student widzi tylko swoje oceny, pracownik dziekanatu zarządza danymi),
- edycja ocen oraz historia zmian,
- przeniesienie filtrowania w wyszukiwarce do zapytania SQL (obecnie lista studentów jest filtrowana w pamięci, co przy dużej liczbie rekordów byłoby nieefektywne) i wyszukiwanie bez rozróżniania wielkości liter,
- migracje Entity Framework zamiast `EnsureCreated()`, aby zmiany schematu nie wymagały usuwania bazy,
- eksport ocen do pliku PDF lub arkusza kalkulacyjnego.

Spis tabel

1	Wymagania funkcjonalne aplikacji	3
2	Porównanie istniejących rozwiązań z projektowaną aplikacją	5
3	Technologie wykorzystane w projekcie	6
4	Zapytania i komendy w projekcie	8
5	Tabela Students	9
6	Tabela Courses	10
7	Tabela Grades	10
8	Scenariusze testowe	19

Spis rysunków

1	Architektura aplikacji Wirtualny Dziekanat	7
2	Diagram związków encji bazy danych	9
3	Diagram sekwencji – wystawienie oceny	14
4	Strona główna ze statystykami	15
5	Lista studentów	15
6	Wyniki wyszukiwania studentów dla frazy „Ni”	15
7	Oceny wybranego studenta ze średnią	16
8	Formularz edycji studenta	16
9	Komunikaty walidacji przy dodawaniu studenta	17
10	Lista przedmiotów	17
11	Lista wystawionych ocen	18
12	Formularz wystawiania oceny	18

Bibliografia

- [1] Microsoft. *Omówienie platformy ASP.NET Core MVC*. URL: <https://learn.microsoft.com/pl-pl/aspnet/core/mvc/overview> (dostęp 26.09.2026).
- [2] Microsoft. *Entity Framework Core – dokumentacja*. URL: <https://learn.microsoft.com/pl-pl/ef/core/> (dostęp 26.09.2026).
- [3] Microsoft. *Wzorzec CQRS – Centrum architektury platformy Azure*. URL: <https://learn.microsoft.com/pl-pl/azure/architecture/patterns/cqrs> (dostęp 26.09.2026).
- [4] Międzyuniwersyteckie Centrum Informatyzacji. *USOS – Uniwersytecki System Obsługi Studiów*. URL: <https://usos.edu.pl> (dostęp 26.09.2026).